# CSE Template

Rohan Mukherjee

November 15, 2024

### 1.1.3

1. I think the main reason cosine similarity works better than other distance metrics like L1/L2 is because it is norm-invariant. If you have two sentence embeddings $x$ and $y$, and these were really similar, one would expect that $2x$ and $2y$ would be really similar too. In fact, you would think these would be just as similar as $x$ and $y$. This is only true for cosine similarity.

2. It chooses option a, pessimistic, instead. Recall that the Glove representation we are using for words is the context based wordvec model. Although positive and sanguine are synonyms, it is more likely that pessimistic has a context more simliar to sanguine than to pessimistic. Of course these words are similar, being antonyms, and there is no way to tell the difference between synonyms and antonyms with Glove.

3. I created the following questions:

   **Question 1:** Relation: Synonyms
   Car is to automobile as bike is to _____?

   a) horse
   b) $\boxed{\text{bicycle}}$
   c) house
   d) oranges

   **Question 2:** Relation: Synonyms
   House is to residence as cow is to _____?

   a) bus

b) horse

c) air

d) bull

**Question 3:** Relation: Antonyms
Happy is to sad as good is to _____?

a) bad

b) horse

c) water

d) cow

**Question 4:** Relation: Antonyms
Big is to small as fast is to _____?

a) slow

b) horse

c) water

d) feline

**Question 5:** Relation: Taxonomy (Basic/Subordinate)
Car is to vehicle as apple is to _____?

a) fruit

b) horse

c) water

d) air

**Question 6:** Relation: Taxonomy (Basic/Subordinate)
Horse is to animal as chair is to _____?

a) fruit

b) furniture

c) water

d) air

4. First, here is the graph of the effect sizes vs vector sizes with the number of tokens fixed at 6B:

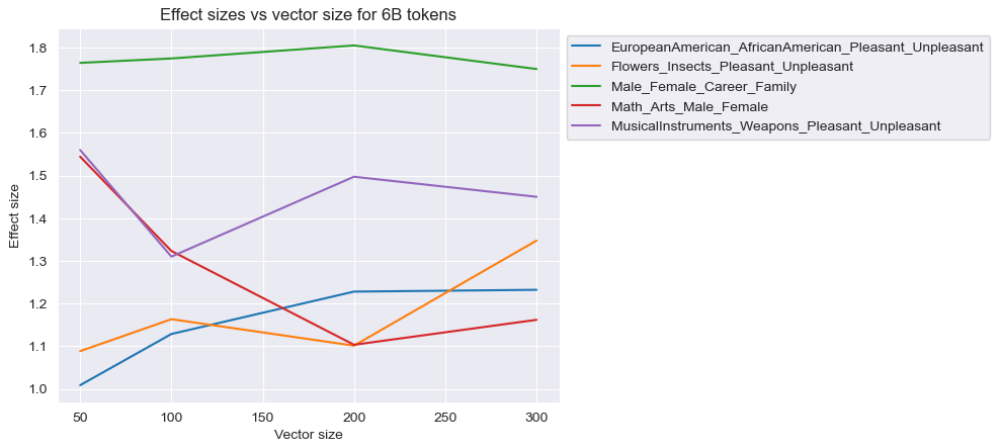



Figure 1: Effect size vs Vector size

Here is the graph of the effect sizese vs number of tokens with vector size fixed at 300:

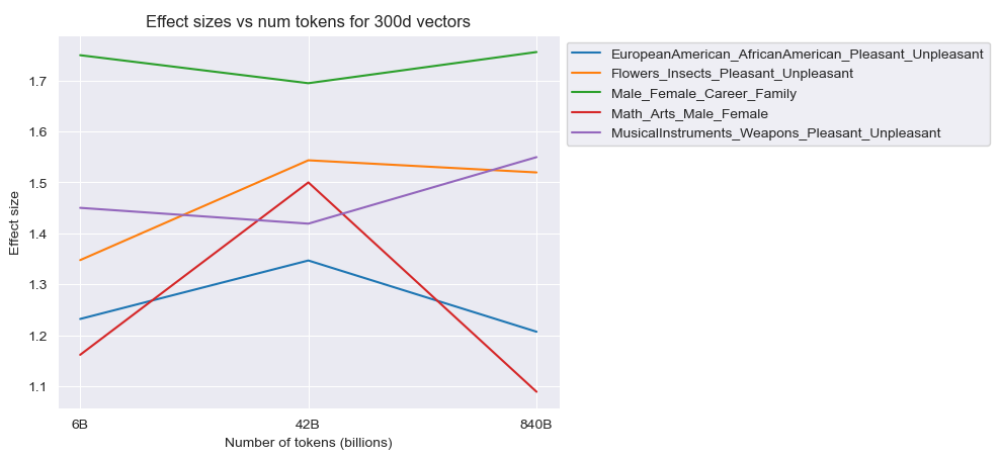



Figure 2: Effect size vs Number of tokens

For both vector sizes and the number of tokens, the male female career family is held basically constant, same with musical instruments weapons. For math arts male female, the effect size drops as both the number of tokens and vector size increase. However, weirdly enough the middle token size of 42B it goes sharply up, then back sharply down. For the other two, they seem to increase at a slow rate.

### 1.2.2

1. The main problem is not understanding the context of our words. This model does not have any sense of the position of the words, so we could do "This movie is great and everyone who says otherwise is totally wrong, and abhorrent!". With no understanding of context this will fail drastically. Similarly, someone could say something like "This movie is great, but the second actor was bad". It is likely this would have a similar sum to "This movie is bad, but the second actor is great".

2. Here is the graph of k vs accuracies:

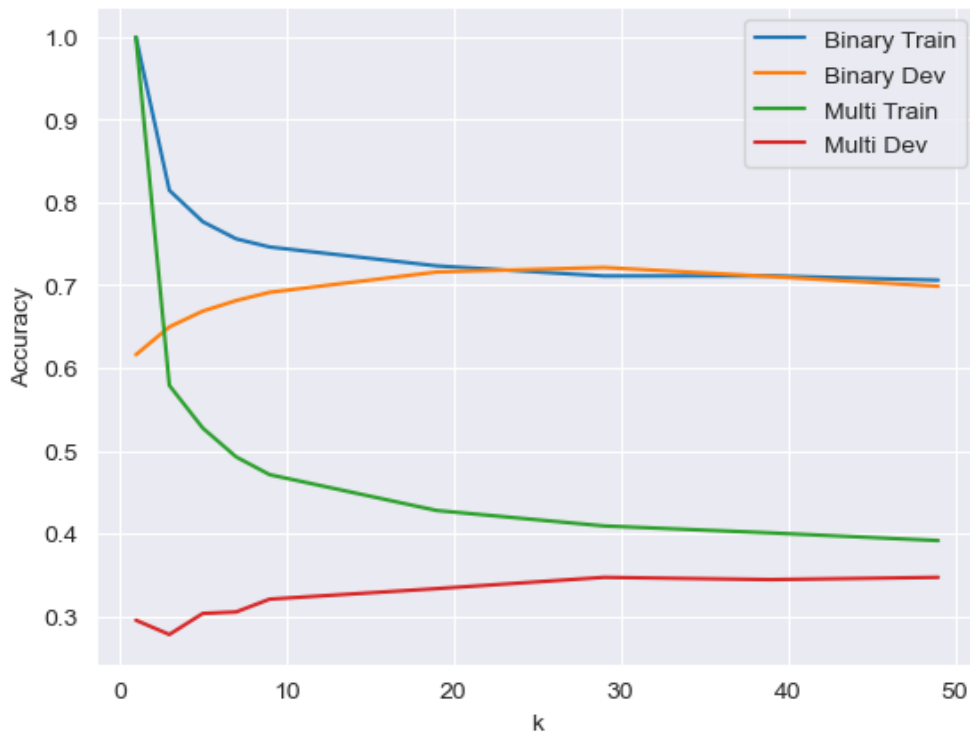



Figure 3: k vs Accuracy

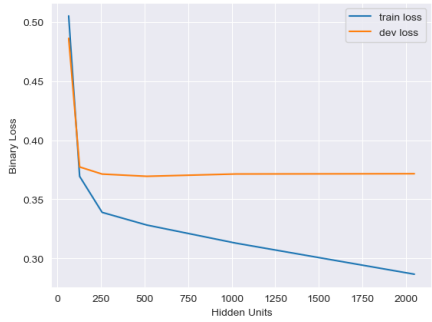
I notice that training accuracy moves consistently down, but the dev accuracy seems to increase up to a point. This is clear, as with k = 1 the model will just predict the same label because it has memorized the training data, and will do very bad on the dev data, as it hasn't. When k increases you get more generalizability, so the dev accuracy increases. But after a while, it gets gradually worse, as taking k to be the entire training set would always just output the same label. You would start comparing it to things that are not relevant.

## 2.2

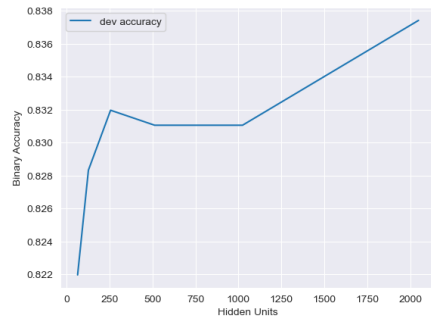
1. Say we set a max number of words of 2048. Then if we used padding instead of a sentence transformer, for the overwhemling majority of sentences, most of the words word be padding. This would mean that the overwhelming majority of our parameters would be useless most of the time, and the few times they are used they haven't been trained enough. So you would see a huge case of overfitting and poor generalizability.

2. Some of the issues I raised above happen when using the Glove representations vs sentence transformers. One notable example is: "And if you're not nearly moved to tears by a couple of scenes, you've got ice water in your veins." This follows almost the exact framework I said above: "This movie is (postiive word) and if you disagree you are (negative word)". Of course the Glove reps would be confused because this is the same sentence as "This movie is (negative word) and if you disagree you are (positive word)". The sentence transformers would be able to understand the context of the words and would be able to differentiate between the two. THe seame logic applies to "The film serves as a valuable time capsule to remind us of the devastating horror suffered by an entire people ."

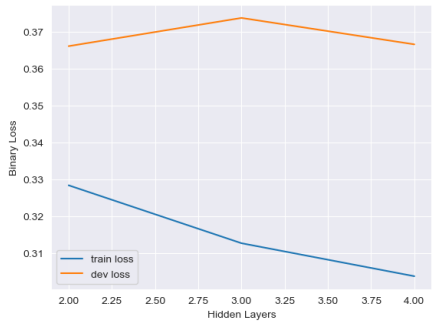
3. Here is a comparison of binary and multi models with different configurations of units and layers for loss and accuracy.



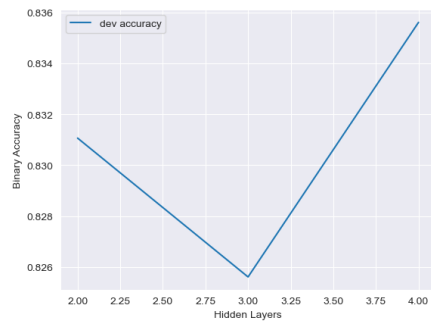

(a) Binary - Units - Loss



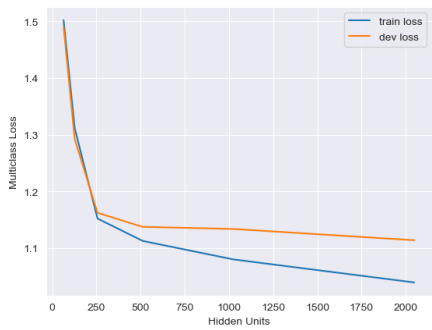

(b) Binary - Units - Accuracy



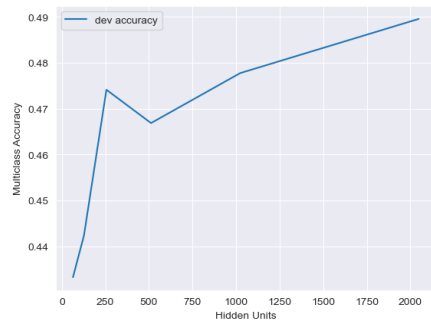

(c) Binary - Layers - Loss



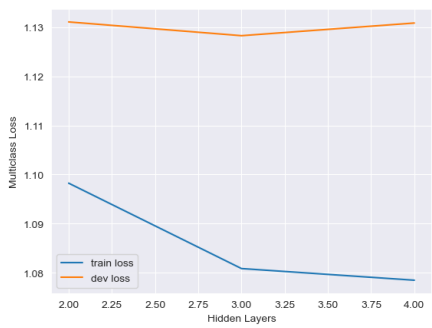

(d) Binary - Layers - Accuracy



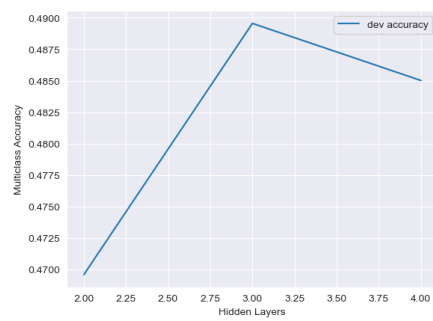

(e) Multi - Units - Loss




(f) Multi - Units - Accuracy




(g) Multi - Layers - Loss

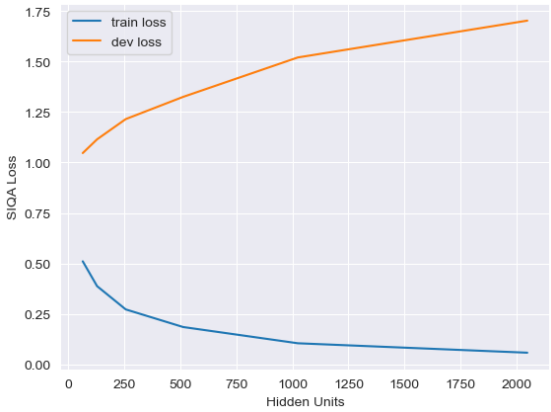



(h) Multi - Layers - Accuracy

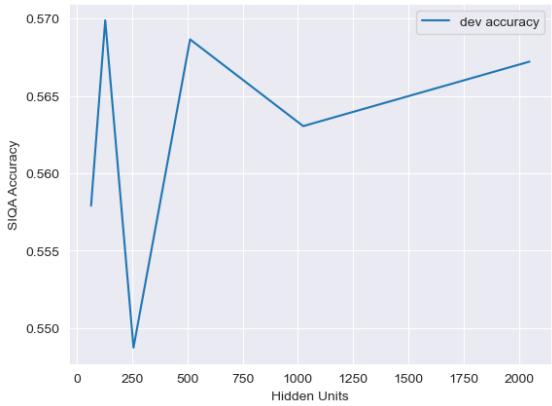
We observe on both parts that generally, having more units up to a point decreases

dev loss and increases accuracy. With the hidden units too low, the linear layers are just not complex enough to understand the problem at hand. However, the model depth seems to just not work very well (in the binary case, the loss is confusing and the accuracy gains are very small on the order of 1%). Using the default parameters, I believe this is probably related to the fact that the number of epochs is too small, or maybe the learning rate too large (since the gradients could start to grow very large as the number of layers increase). When I tried increasing the number of epochs, I realized that it was actually overfitting with 4 layers. This is pretty surprising to me! Most of the time 1 hidden layer got the best results.

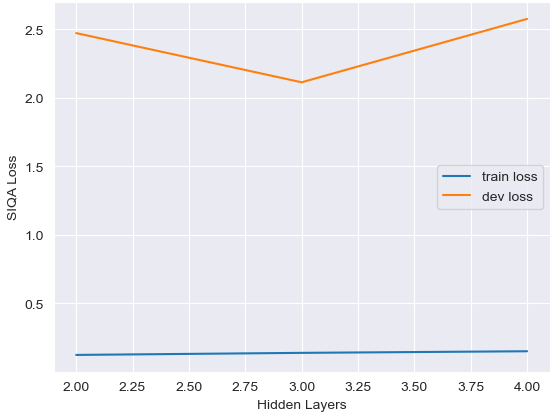
Here is the graph for SIQA: Here we see a catastrophic case of overfitting for both



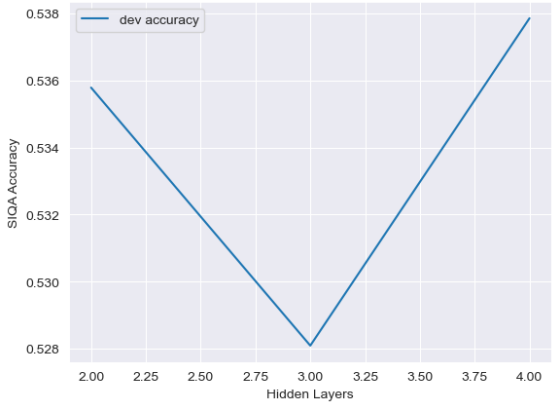

(a) SIQA - Units - Loss




(b) SIQA - Units - Accuracy




(c) SIQA - Layers - Loss




(d) SIQA - Layers - Accuracy

Figure 5: SIQA results comparing units and layers configurations for loss and accuracy.

models. The loss dips lower and lower for the training data, yet the dev loss and accuracy are all over the place. The same happens with the number of layers, but surprisingly, the training loss stays at about the same place. When I trained on 40

epochs, the training loss dipped to 0.06, while the dev loss skyrocketed to 2.9. This is clear case of overfitting.